

Vidya Academy of Science & Technology

(A unit of Vidya International Charitable Trust)

Approved by AICTE & Affiliated to APJ Abdul Kalam Technological University



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CSL 332 NETWORKING LAB

LAB MANUAL

PREPARED BY:

Mr. SIVADASAN E T

APPROVED BY:

HOD, CSE

VISION OF THE INSTITUTION

Progress through education

MISSION OF THE INSTITUTION

To seek, strive for and scale greater heights of quality education

VISION OF THE DEPARTMENT

Progress through quality education and become a centre of excellence in computing knowledge

MISSION OF THE DEPARTMENT

To seek, to strive for advancements in technological and social aspects so as to create socially responsible engineers in the field of computer science, who will embark to further reduce gap between science and man-kind.

PROGRAM EDUCATIONAL OBJECTIVES (PEOS)

PEO-1: Graduates shall practice Computer Science and Engineering in IT industry, public sector undertaking, as an entrepreneur or as a researcher pursuing a successful professional career.

PEO-2: Graduates shall demonstrate creativity, develop innovative ideas and work in teams to accomplish a common goal.

PEO-3: Graduates shall demonstrate project management skills, lifelong learning and social awareness upholding ethical values.

PROGRAM SPECIFIC OUTCOMES (PSOS)

PSO-1: The ability to apply the knowledge of computational structures and algorithms for solving complex engineering problems in multidisciplinary areas.

PSO-2: The ability to design, develop and evaluate computer based systems using advanced tools and technologies to meet the real world challenges.

PROGRAM OUTCOMES (POS)

Computer Science & Engineering graduates will be able to:

PO-1: Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and

Computer Science & Engineering specialization to the solution of complex engineering problems.

PO-2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO-3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO-4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO-5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO-6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO-7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO-8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO-9: Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary surroundings.

PO-10:Communication: Communicate effectively on complex engineering activities with the engineering

community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO-11:Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO-12: Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

COURSE SYLLABUS**LAB Experiments:**

1. Getting started with the basics of network configuration files and networking commands in Linux.*
2. To familiarize and understand the use and functioning of system calls used for network programming in Linux.*
3. Implement client-server communication using socket programming and TCP as transport layer protocol*
4. Implement client-server communication using socket programming and UDP as transport layer protocol*
5. Simulate sliding window flow control protocols.* (Stop and Wait, Go back N, Selective Repeat ARQ protocols)
6. Implement and simulate algorithm for Distance Vector Routing protocol or Link State Routing protocol.*
7. Implement Simple Mail Transfer Protocol.
8. Implement File Transfer Protocol.*
9. Implement congestion control using a leaky bucket algorithm.*
10. Understanding Wire shark tool.*
11. Design and configure a network with multiple subnets with wired and wireless LANs using required network devices. Configure commonly used services in the network.*
12. Study of NS2 simulator*

COURSE OUTCOMES

At the end of the course students should be able to:

Course Outcomes:

CO#	Course Outcomes
CO1	Use network related commands and configuration files in Linux Operating System. (Cognitive Knowledge Level: Understand).
CO2	Develop network application programs and protocols. (Cognitive Knowledge Level: Apply)
CO3	Analyze network traffic using network monitoring tools. (Cognitive Knowledge Level: Apply)
CO4	Design and setup a network and configure different network protocols. (Cognitive Knowledge Level: Apply)
CO5	Develop simulation of fundamental network concepts using a network simulator. (Cognitive Knowledge Level: Apply)

CO-PO/PSO Mapping

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	3	3	3					2		2		2	2	
CO2	3	3	3	2				2		2		2	2	2
CO3	3	3	3	2	2			2		2		2	2	2
CO4	3	3	3	2	2	2		2		2		2	2	
CO5	3	3	3	2	5			2		2		2	2	

GENERAL INSTRUCTIONS (For the Lab)**Preparation**

- Understand the objective and networking concept of the experiment.
- Study the required socket functions and system calls in C.
- Review sample client–server programs before implementation.
- Ensure familiarity with TCP/UDP communication models.

Lab Setup

- Use a system with a C compiler (gcc) and Linux/UNIX environment.
- Ensure necessary header files for socket programming are available.
- Arrange terminals or systems for client and server execution.
- Check network connectivity and firewall permissions if required.

Documentation

- Write clear comments explaining logic and function usage.
- Maintain a lab record with aim, algorithm, program code, and output.
- Record observations and errors encountered during execution.
- Include screenshots or terminal outputs where required.

Coding Standard

- Follow proper indentation and readable formatting.
- Use meaningful variable and function names.
- Handle errors for all system calls.
- Avoid hard-coding values; use constants where possible.
- Keep client and server code modular and well-structured.

Testing & Debugging

- Compile code without warnings using gcc.
 - `gcc -o server server.c`
 - `gcc -o client client.c`
- Run the server program on the remote server: `./server`
- Run the client program on the same or another machine: `./client`
- Test programs with valid and invalid inputs.
- Verify proper data transmission between client and server.
- Use debugging tools (`printf`, `gdb`) to trace errors.
- Ensure all sockets are closed properly after execution.

INDEX

Sl No	Experiment Name	Page No
1	Getting started with basics of network configurations files and networking commands in linux	9
2	To familiarize and understand the use and functioning of system calls used for network programming in Linux	12
3	Simple TCP/IP Client Server Program	18
4	Concurrent UDP Time Server	25
5	Simulate sliding window flow control protocols (Stop and Wait, Go Back N, Selective Repeat ARQ protocols)	29
6	Distance Vector	52
7	File Transfer Protocol	58
8	Simple Mail Transfer Protocol	64
9	Leaky Bucket	77
10	Understanding the Wireshark Tool	81
11	Design and configure network with multiple subnets using required network devices	83
12	Study of NS2 Simulator	86

EXP NO: 1**GETTING STARTED WITH BASICS OF NETWORK CONFIGURATIONS
FILES AND NETWORKING COMMANDS IN LINUX.****AIM**

To familiarize Network configuration files and networking commands in Linux operating system.

LEARNING OUTCOME

- Understand the purpose and structure of network configuration files in the Linux operating system.
- Use basic and advanced networking commands to view and manage network settings.
- Configure IP addresses, gateways, and DNS settings using Linux tools.
- Verify and troubleshoot network connectivity

The important network configuration files in Linux operating systems are

1. /etc/hosts

This file is used to resolve hostnames on small networks with no DNS server. This text file contains a mapping of an IP address to the corresponding host name in each line. This file also contains a line specifying the IP address of the loopback device i.e, 127.0.0.1 is mapped to localhost.

A typical hosts file is as shown

```
127.0.0.1 localhost
```

```
127.0.1.1 anil-300E4Z-300E5Z-300E7Z
```

2. /etc/resolv.conf

This configuration file contains the IP addresses of DNS servers and the search domain.

A sample file is shown

```
# DO NOT EDIT THIS FILE BY HAND -- YOUR CHANGES WILL BE OVERWRITTEN
```

```
nameserver 127.0.1.1
```

3. **/etc/sysconfig/network**

This configuration file specifies routing and host information for all network interfaces. It contains directives that are global specific. For example if NETWORKING=yes, then /etc/init.d/network activates network devices.

4. **/etc/nsswitch.conf**

This file includes database search entries. The directive specifies which database is to be searched first.

The important Linux networking commands are

1. **ifconfig**

This command gives the configuration of all interfaces in the system. It can be run with an interface name to get the details of the interface. `ifconfig wlan0` Link encap:Ethernet HWaddr b8:03:05:ad:6b:23 inet addr:192.168.43.15 Bcast:192.168.43.255 Mask:255.255.255.0 inet6 addr: 2405:204:d206:d3b1:ba03:5ff:fead:6b23/64 Scope:Global inet6 addr:fe80::ba03:5ff:fead:6b23/64 Scope:Link inet6 addr: 2405:204:d206:d3b1:21ee:5665:de59:bd4e/64 Scope:Global UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1 RX packets:827087 errors:0 dropped:0 overruns:0 frame:0 TX packets:433391 errors:0 dropped:0 overruns:0 carrier:0 collisions:0 txqueuelen:1000 RX bytes:1117797710 (1.1 GB) TX bytes:53252386 (53.2 MB)

This gives the IP address, subnet mask, and broadcast address of the wireless LAN adapter. Also tells that it can support multicasting. If eth0 is given as the parameter, the command gives the details of the Ethernet adapter.

2. **netstat**

This command gives network status information.

Netstat -i

```

Iface MTU Met RX-OK RX-ERR RX-DRP RX-OVR TX-OK TX-ERR TX-DRP TX-OVR Flg

```

As shown above, the command with -i flag provides information on the interfaces. lo stands for loopback interface.

3. ping

This is the most commonly used command for checking connectivity.

```
ping www.google.com
```

```
PING www.google.com (172.217.163.36) 56(84) bytes of data.
```

```
64 bytes from maa05s01-in-f4.1e100.net (172.217.163.36): icmp_seq=1 ttl=53 time=51.4 ms 64
bytes from maa05s01-in-f4.1e100.net (172.217.163.36): icmp_seq=2 ttl=53 time=50.3 ms 64 bytes
from maa05s01-in-f4.1e100.net (172.217.163.36): icmp_seq=3 ttl=53 time=48.5 ms 64 bytes from
maa05s01-in-f4.1e100.net (172.217.163.36): icmp_seq=4 ttl=53 time=59.8 ms 64 bytes from
maa05s01-in-f4.1e100.net (172.217.163.36): icmp_seq=5 ttl=53 time=57.8 ms 64 bytes from
maa05s01-in-f4.1e100.net (172.217.163.36): icmp_seq=6 ttl=53 time=59.2 ms 64 bytes from
maa05s01-in-f4.1e100.net (172.217.163.36): icmp_seq=7 ttl=53 time=68.2 ms 64 bytes from
maa05s01-in-f4.1e100.net (172.217.163.36): icmp_seq=8 ttl=53 time=58.8 ms
```

```
^C
```

```
--- www.google.com ping statistics ---
```

```
8 packets transmitted, 8 received, 0% packet loss, time 7004ms rtt eth0 1500 0 0 0 0 0 0 0 0 BMU
lo 65536 0 12166 0 0 0 12166 0 0 0 LRU wlan0 1500 0 827946 0 0 0 434246 0 0 0 BMRU
```

MODEL VIVA QUESTIONS

1. Name important network configuration files in Linux.
2. What is the use of the ifconfig command?
3. What is the purpose of the ping command?
4. How can you check the IP address of a system in Linux?
5. What is a gateway?

RESULT

The basics of Linux network configuration files and networking commands were studied and executed successfully.

EXP NO: 2**TO FAMILIARIZE AND UNDERSTAND THE USE AND
FUNCTIONING OF SYSTEM CALLS USED FOR
NETWORK PROGRAMMING IN LINUX****AIM**

To familiarize and understand the use and functioning of system calls used for network programming in Linux.

LEARNING OUTCOME

- Understand the role of system calls in network programming.
- Use network-related system calls in Linux.
- Develop basic client–server programs using C.
- Analyze data communication between processes.

System Calls in Network Programming

System calls are low-level functions that interact directly with the kernel, enabling user applications to communicate with hardware or perform networking tasks like creating sockets, sending/receiving data, and managing connections. Let's break down the key network programming functions and explain the parameters used in each.

1. socket()

Creates an endpoint for communication.

```
int socket(int domain, int type, int protocol);
```

1. domain – Specifies the protocol family (addressing type).
 1. AF_INET – IPv4
 2. AF_INET6 – IPv6
 3. AF_UNIX – Local socket (inter-process communication)
2. type – Defines the communication semantics.
 1. SOCK_STREAM – TCP (connection-oriented)

2. SOCK_DGRAM – UDP (connectionless)

3. protocol – Protocol to be used.

1. 0 – Automatically selects the default protocol (e.g., TCP for SOCK_STREAM).

2. IPPROTO_TCP – TCP

3. IPPROTO_UDP – UDP

Return Value:

- A file descriptor for the socket. On error, returns -1.

2. bind()

Assigns an address to a socket.

```
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

- sockfd – The socket file descriptor returned by socket().
- addr – A pointer to sockaddr structure specifying the address to bind.

```
struct sockaddr_in addr;
```

```
addr.sin_family = AF_INET;
```

```
addr.sin_addr.s_addr = INADDR_ANY;
```

```
addr.sin_port = htons(8080);
```

- addrlen – Size of the addr structure, typically sizeof(addr).

Return Value:

- 0 on success, -1 on error.

3. listen()

Marks the socket as passive, waiting for connections.

```
int listen(int sockfd, int backlog);
```

- sockfd – The bound socket descriptor.
- backlog – Maximum number of pending connections in the queue.
 - Typical values: 5 or 10.

Return Value:

- 0 on success, -1 on error.

4. accept()

Accepts incoming connection requests.

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

- sockfd – Listening socket descriptor.
- addr – Pointer to a sockaddr structure to store the client address.
- addrlen – Size of the addr structure.
 - On return, it contains the size of the client address.

Return Value:

- A new socket descriptor for the connection.

5. connect()

Initiates a connection to a remote server.

```
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

- sockfd – The socket descriptor created by socket().
- addr – Pointer to the sockaddr structure representing the server's address.
- addrlen – Size of the addr structure.

Return Value:

- 0 on success, -1 on error.

6. send()

Sends data through the socket.

```
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
```

- sockfd – The socket descriptor.
- buf – Pointer to the data to be sent.
- len – Length of data in bytes.
- flags – Modifies send behavior (usually 0).

Return Value:

- Number of bytes sent, -1 on error.

7. recv()

Receives data from the socket.

```
ssize_t recv(int sockfd, void *buf, size_t len, int flags);
```

- sockfd – The socket descriptor.
- buf – Buffer to store received data.
- len – Maximum size of the buffer.
- flags – Modifies receive behavior (0 for default).

Return Value:

- Number of bytes received, -1 on error.

8. close()

Closes the socket and releases resources.

```
int close(int sockfd);
```

- sockfd – The socket file descriptor.

Return Value:

- 0 on success, -1 on error.

9. setsockopt()

Configures socket options.

```
int setsockopt(int sockfd, int level, int optname, const void *optval, socklen_t optlen);
```

- sockfd – The socket descriptor.
- level – Protocol level (SOL_SOCKET for general socket options).
- optname – Option to set (e.g., SO_REUSEADDR to reuse local addresses).
- optval – Pointer to the value to set.
- optlen – Size of the optval.

Return Value:

- 0 on success, -1 on error.

10. getsockopt()

Retrieves socket options.

```
int getsockopt(int sockfd, int level, int optname, void *optval, socklen_t *optlen);
```

- sockfd – The socket descriptor.
- level – Protocol level (SOL_SOCKET).
- optname – Option to retrieve.
- optval – Buffer to store option value.
- optlen – Size of the buffer.

Return Value:

- 0 on success, -1 on error.

Example Breakdown

```
sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

- AF_INET – IPv4
- SOCK_STREAM – TCP (connection-oriented)
- 0 – Default protocol (TCP for stream sockets)

```
bind(sockfd, (struct sockaddr *)&addr, sizeof(addr));
```

- sockfd – Socket to bind.
- addr – Address structure containing IP and port.
- sizeof(addr) – Size of the address structure.

```
listen(sockfd, 5);
```

- 5 – Queue length for pending connections.

```
newsock = accept(sockfd, (struct sockaddr *)&client, &len);
```

- client – Fills with the client's address.
- len – Length of the client structure.

Common Network-Related System Calls

System Call	Description	Example Usage
<code>socket()</code>	Creates a socket (communication endpoint).	<code>sockfd = socket(AF_INET, SOCK_STREAM, 0);</code>
<code>bind()</code>	Binds socket to an IP/port.	<code>bind(sockfd, (struct sockaddr *)&addr, sizeof(addr));</code>
<code>listen()</code>	Listens for incoming connections (server-side).	<code>listen(sockfd, 5);</code>
<code>accept()</code>	Accepts incoming connection requests.	<code>newsock = accept(sockfd, (struct sockaddr *)&client, &len);</code>
<code>connect()</code>	Connects to a remote socket (client-side).	<code>connect(sockfd, (struct sockaddr *)&server, sizeof(server));</code>
<code>send()</code>	Sends data to a socket.	<code>send(sockfd, message, strlen(message), 0);</code>
<code>recv()</code>	Receives data from a socket.	<code>recv(sockfd, buffer, sizeof(buffer), 0);</code>
<code>close()</code>	Closes a socket.	<code>close(sockfd);</code>
<code>getsockopt()</code>	Retrieves socket options.	<code>getsockopt(sockfd, SOL_SOCKET, SO_ERROR, &error, &len);</code>
<code>setsockopt()</code>	Modifies socket options.	<code>setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));</code>

MODEL VIVA QUESTIONS

1. What is a system call?
2. Why are system calls required in network programming?
3. What is a socket?
4. What is a port number?
5. Difference between TCP and UDP.

RESULT

The use and functioning of system calls required for network programming in Linux were successfully studied and understood.

EXP NO: 3**SIMPLE TCP/IP CLIENT SERVER PROGRAM****AIM**

To design and implement a simple TCP/IP client-server program that establishes a connection between a client and a server and enables data communication using sockets.

LEARNING OUTCOME

- Learn how to create and configure TCP sockets for client and server applications.
- Implement a client-server model for reliable data transmission.
- Analyze the connection establishment and termination process in TCP.
- Gain hands-on experience in sending and receiving data over a network.
- Develop basic debugging skills for network-based applications.

DESCRIPTION**Socket Programming in C**

Socket programming is a method used in network communication that allows data to be sent and received between devices. It is a critical concept in computer networking and is widely used to develop client-server applications. the fundamentals of socket programming in C, focusing on TCP (Transmission Control Protocol) for reliable, connection-oriented communication.

Socket programming enables communication between two nodes on a network. A server listens for incoming client requests, and the client connects to the server to facilitate data exchange. Sockets provide a communication channel between two processes, either on the same machine or different machines connected via a network.

Types of Sockets**1. Stream Sockets (SOCK_STREAM):**

- Uses TCP for communication.
- Provides reliable, connection-oriented communication.

- Data is transmitted in order and without loss.

2. Datagram Sockets (SOCK_DGRAM):

- Uses UDP (User Datagram Protocol).
- Connectionless and unreliable, but faster.
- Data may arrive out of order or get lost.

3. Raw Sockets:

- Allows direct access to lower-level protocols.
- Used for packet-level manipulation.

ALGORITHM

Server

1. Initialize variables and create buffer.
2. Create TCP socket using socket().
3. Set server address family, IP, and port.
4. Bind socket to specified port using bind().
5. Set socket to listening mode using listen().
6. Print “Waiting for connection...”.
7. Accept a client request using accept().
8. Read data from the client.
9. Print the received message.
10. Send a reply to the client.
11. Close client socket.
12. Close server socket.
13. Terminate program.

Client

1. Start the program.
2. Declare socket variable, server structure, and message buffer.

3. Create a TCP socket using `socket()`.
4. Set server IP address and port number (127.0.0.1, 8080).
5. Connect to the server using `connect()`.
6. Send message "Hello Server" to the server.
7. Receive server response using `read()`.
8. Display the received response.
9. Close the socket connection.
10. End the program.

PROGRAM

Server

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#include<netdb.h>
int main(){
    char buf[100];
    int k, sockfd,temp;
    struct sockaddr_in server,client;
    socklen_t len;
    memset(&server,0,sizeof(server));
    memset(&client,0,sizeof(client));
    sockfd = socket(AF_INET,SOCK_STREAM,0);
    if(sockfd == -1){
        printf("Error in socket creation\n");
        exit(1);
    }
    server.sin_family = AF_INET;
    server.sin_port = 3000;
    server.sin_addr.s_addr = INADDR_ANY;
    k = bind(sockfd,(struct sockaddr*)&server,sizeof(server));
    if(k == -1){
        printf("Error in binding\n");
```

```
    exit(1);
}
k = listen(sockfd,20);
if(k == -1){
    printf("Error in listening\n");
    exit(1);
}
len = sizeof(client);
temp = accept(sockfd,(struct sockaddr*)&client, &len);
if(temp == -1){
    printf("Error in temporary socket creation\n");
    exit(1);
}
while(1){
    k = recv(temp,buf,100,0);
    if(k == -1){
        printf("Error in receiving\n");
        exit(1);
    }
    printf("Message got from client is : %s",buf);
    printf("Enter the data to be send to client : ");
    fgets(buf,100,stdin);
    if(strncmp(buf,"end",3)==0)
        break;
    k = send(temp,buf,100,0);
    if(k == -1){
        printf("Error in sending\n");
        exit(1);
    }
}
close(temp);
return 0;
}
```

Client

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#include<netdb.h>
int main()
{
    char buf[100];
    int k, sockfd;
    struct sockaddr_in client;
    sockfd = socket(AF_INET,SOCK_STREAM,0);
    if(sockfd == -1){
        printf("Error in socket creation\n");
        exit(1);
    }
    client.sin_family = AF_INET;
    client.sin_port = 3000;
    client.sin_addr.s_addr = INADDR_ANY;
    k = connect(sockfd,(struct sockaddr*)&client,sizeof(client));
    if(k == -1){
        printf("Error in connecting to server\n");
        exit(1);
    }
    while(1){
        printf("Enter the data to be send to server : ");
        fgets(buf,100,stdin);
        if(strncmp(buf,"end",3)==0)
            break;
        k = send(sockfd,buf,100,0);
        if(k == -1){
            printf("Error in sending\n");
            exit(1);
        }

        k = recv(sockfd,buf,100,0);
```

```
        if(k == -1){
            printf("Error in receiving\n");
            exit(1);
        }

        printf("Message got from server is : %s",buf);
    }
    close(sockfd);
    return 0;
}
```

EXPECTED OUTPUT

\$./server

Message got from client is : hello server

Enter the data to be send to client : hi client

Message got from client is : how are you?

Enter the data to be send to client : I am fine

Message got from client is : end

\$./client

Enter the data to be send to server : hello server

Message got from server is : hi client

Enter the data to be send to server : how are you?

Message got from server is : I am fine

Enter the data to be send to server : end

MODEL VIVA QUESTIONS

What is TCP/IP?

What is a socket?

What is the purpose of bind()?

What is localhost?

What does accept() do?

RESULT

The simple TCP/IP client-server communication using TCP was implemented and verified successfully.

EXP NO: 4

CONCURRENT UDP TIME SERVER

AIM

To implement concurrent Time server using UDP

LEARNING OBJECTIVE

- Understand the working principles of the UDP protocol and its connectionless communication model.
- Learn the concept of concurrency in network servers.
- Design and implement a concurrent UDP server to handle multiple client requests simultaneously.
- Analyze the differences between concurrent and iterative server models.
- Evaluate the advantages and limitations of UDP in real-time and lightweight communication.
- Gain practical experience in handling datagrams, client addressing, and synchronization issues.

DESCRIPTION

A **Concurrent UDP Time Server** is a server application that uses the **UDP protocol** to provide the current time to multiple clients simultaneously. UDP (User Datagram Protocol) socket programming is a method of communication between devices over a network using the UDP protocol. Unlike TCP, UDP is a **connectionless** and **lightweight** protocol. It sends data as independent packets called **datagrams** without establishing a dedicated connection between sender and receiver.

UDP is commonly used in applications where **speed is more important than reliability**, such as online gaming, video streaming, voice calls, and real-time applications.

ALGORITHM**Server**

1. Start the program.
2. Create a UDP socket using socket().
3. Set server IP address and port number in server_addr.
4. Send "TIME_REQUEST" to the server using sendto().
5. Wait for the server's reply using recvfrom().
6. Display the received time.
7. Close the socket.
8. End the program.

Client

1. Start the program.
2. Create a UDP socket.
3. Set server address (IP = 127.0.0.1, Port = 12345).
4. Send "TIME_REQUEST" to the server using sendto().
5. Receive the server's response using recvfrom().
6. Display the received time.
7. Close the socket.
8. End the program.

PROGRAM**Server**

```
#include<stdio.h>
#include<string.h>
#include<netdb.h>
#include<time.h>
int main(){
```

```
char buf[1024];
time_t ct;
struct sockaddr_in server,client;
int sockfd = socket(AF_INET,SOCK_DGRAM,0);
server.sin_family = AF_INET;
server.sin_port = 7777;
server.sin_addr.s_addr = INADDR_ANY;
bind(sockfd,(struct sockaddr*)&server,sizeof(server));
int len = sizeof(client);
while(1){
    recvfrom(sockfd,buf,sizeof(buf),0,(struct sockaddr*)&client,&len);
    printf("Time request from client\n");
    ct = time(NULL);
    sprintf(buf,"%s",ctime(&ct));
    if(fork()==0){
        while(1){
            sendto(sockfd,buf,sizeof(buf),0,(struct
sockaddr*)&client,sizeof(client));
            memset(buf,0,1024);
            recvfrom(sockfd,buf,sizeof(buf),0,(struct sockaddr*)&client,&len);
            printf("Time request from client\n");
            memset(buf,0,1024);
            sprintf(buf,"%s",ctime(&ct));
        }
    }
}
close(sockfd);
}
```

Client

// Client Code

```
#include<stdio.h>
#include<string.h>
#include<netdb.h>
int main(){
    char buf[1024];
```

```
struct sockaddr_in server;
server.sin_family = AF_INET;
server.sin_port = 7777;
server.sin_addr.s_addr = INADDR_ANY;
int sockfd = socket(AF_INET,SOCK_DGRAM,0);
bind(sockfd,(struct sockaddr*)&server,sizeof(server));
sprintf(buf,"%s","time");
sendto(sockfd,buf,1024,0,(struct sockaddr*)&server,sizeof(server));
int len = sizeof(server);
recvfrom(sockfd,buf,1024,0,(struct sockaddr*)&server,&len);
printf("Current time is : %s",buf);
close(sockfd);
}
```

EXPECTED OUTPUT

- **Server:**

Time request from client

- **Client:**

Current time is : Thu Dec 4 15:26:17 2025

MODEL VIVA QUESTIONS

1. What is a UDP time server?
2. Is listen() required in a UDP server?
3. Which socket type is used in a UDP server?
4. How does the server know the client's address?
5. What is the limitation of UDP?

RESULT

The concurrent UDP time server was executed successfully.

EXP NO: 5

SIMULATE SLIDING WINDOW FLOW CONTROL PROTOCOLS.

AIM

To Simulate sliding window flow control protocols. (Stop and Wait, Go back N, Selective Repeat ARQ protocols)

LEARNING OBJECTIVE

- To understand the need for flow control and error control in data communication.
- To study the working principle of sliding window mechanisms in reliable data transfer.
- To analyze the operation of Stop-and-Wait ARQ and its efficiency limitations.
- To explain the functioning of Go-Back-N ARQ, including pipelining and retransmission strategies.
- To understand Selective Repeat ARQ and its role in improving channel utilization.
- To compare Stop-and-Wait, Go-Back-N, and Selective Repeat ARQ based on throughput, efficiency, and complexity.
- To evaluate the performance of different ARQ protocols under varying network conditions.

DESCRIPTION

Stop & Wait : The **Stop-and-Wait protocol** is a basic and reliable data transmission method used in computer networks to ensure that data is delivered correctly from sender to receiver. It works by allowing the sender to transmit only one data frame at a time and then stopping to wait for an acknowledgment (ACK) from the receiver before sending the next frame. After sending a frame, the sender waits until the receiver confirms successful delivery; if an ACK does not arrive within a specific timeout period, the sender assumes the frame or acknowledgment was lost and retransmits the same frame. Once the correct ACK is received, the sender proceeds with the next frame. This simple process ensures reliable data transfer but is slow because the sender must wait for an acknowledgment after every frame.

Go Back N : The **Go-Back-N (GBN) protocol** is a reliable data transmission protocol used in computer networks that employs sliding window flow control. Unlike Stop-and-Wait, it allows the sender to send multiple frames consecutively (up to a fixed window size, N) without waiting for an acknowledgment for each frame.

The receiver, however, only accepts frames in order. If a frame is lost or received with an error, the receiver discards that frame and all subsequent frames, sending an acknowledgment (ACK) for the last correctly received in-order frame. The sender, upon receiving this acknowledgment or a timeout, retransmits that frame and all subsequent frames in the window.

Selective Repeat ARQ : The **Selective Repeat (SR) Automatic Repeat Request (ARQ)** protocol is a reliable data transmission protocol that uses a sliding window mechanism to improve efficiency compared to Go-Back-N. In this protocol, the sender can transmit multiple frames (up to a defined window size) without waiting for individual acknowledgments. However, unlike Go-Back-N, the receiver accepts frames out of order and stores them temporarily. Only the frames that are lost or received in error are retransmitted by the sender, rather than retransmitting all subsequent frames. This selective retransmission reduces unnecessary data transfer and improves bandwidth utilization, especially in networks with higher error rates.

STOP & WAIT

ALGORITHM

Server

1. Start the program.
2. Create a UDP socket.
3. Set server IP and port.
4. Bind the socket to the server address.
5. Repeat indefinitely:
 1. Receive a frame from the client.
 2. If frame is valid and has expected sequence number:
 3. Print received data.
 4. Randomly decide whether to send ACK.

5. If ACK is sent, increment expected sequence number.

6. Else, print “frame not received or out of order.”

6. Close the socket.

7. End the program.

Client

1. Start the program.

2. Create a UDP socket.

3. Set server IP and port.

4. Initialize frame_id = 0 and ack_received = 1.

5. Repeat indefinitely:

6. If ack_received = 1:

1. Prompt user to enter data for the frame.

2. Send data frame with seq = frame_id.

7. Wait for ACK from the server.

8. If ACK is received and valid:

1. Print acknowledgment received.

2. Increment frame_id and set ack_received = 1.

9. Else (ACK not received or invalid):

1. Print acknowledgment not received.

2. Set ack_received = 0 and retransmit frame after a short delay.

10. Close the socket.

11. End the program.

PROGRAM**Server**

```
server.c
#include<stdio.h>
#include<string.h>
#include<sys/types.h>
#include<netinet/in.h>
#include<arpa/inet.h>
typedef struct packet{
char data[1024];
}Packet;
typedef struct frame{
int kind; //SEQ = 1 , ACK = 0
int sq;
int ack;
Packet packet;
}Frame;
int main(){
int frame_id = 0, i;
Frame frecv;
Frame fsend;
int sockfd = socket(AF_INET,SOCK_DGRAM,0);
struct sockaddr_in server,client;
server.sin_family = AF_INET;
server.sin_port = 8888;
server.sin_addr.s_addr = INADDR_ANY;
bind(sockfd,(struct sockaddr*)&server,sizeof(server));
int len = sizeof(client);
while(1){
recvfrom(sockfd,&frecv,sizeof(Frame),0,(struct sockaddr*)&client,&len);
printf("%d %d %d\n", frecv.kind, frecv.sq, frame_id);
if(frecv.kind == 1 && frecv.sq == frame_id){
i = (rand() %(10 - 0 + 1)) + 0;
```

```
printf("[+] Frame received : %s\n",frecv.packet.data);
fsend.sq = 0;
fsend.kind = 0;
printf("i = %d",i);
fsend.ack = frecv.sq+1;
if(i%3==0){
fsend.packet.data[0] = 'e';
sendto(sockfd,&fsend,sizeof(fsend),0,(struct sockaddr*)&client,len);
printf("[+] Ack not send\n");
}else{
fsend.packet.data[0] = 'c';
sendto(sockfd,&fsend,sizeof(Frame),0,(struct sockaddr*)&client,len);
printf("[+] Ack send\n");
}
}
else
printf("[-] Frame not received\n");
frame_id++;
}
}
```

client

```
#include<stdio.h>
#include<string.h>
#include<sys/types.h>
#include<netinet/in.h>
#include<arpa/inet.h>
typedef struct packet{
char data[1024];
}Packet;
typedef struct frame{
int kind; //SEQ = 1, ACK = 0
int sq;
int ack;
Packet packet;
```

```
}Frame;

int main(){
int frame_id = 0, i=0;
int ack_rcv = 1;
Frame frecv;
Frame fsend;
int sockfd = socket(AF_INET,SOCK_DGRAM,0);
struct sockaddr_in server;
int len = sizeof(server);
server.sin_family = AF_INET;
server.sin_port = 8888;
server.sin_addr.s_addr = INADDR_ANY;
bind(sockfd,(struct sockaddr*)&server,sizeof(server));
while(1){
if(ack_rcv==0){
sleep(1);

fsend.sq = frame_id;
sendto(sockfd,&fsend,sizeof(Frame),0,(struct sockaddr*)&server,sizeof(server));
printf("[+] %s Frame send%d\n",fsend.packet.data,i);
i++;
}
if(ack_rcv==1){
fsend.sq = frame_id;
fsend.kind = 1;
fsend.ack = 0;
printf("Enter the data : ");
scanf("%s",fsend.packet.data);
sendto(sockfd,&fsend,sizeof(Frame),0,(struct sockaddr*)&server,sizeof(server));
printf("[+] Frame send %d\n",i);
i++;
}
recvfrom(sockfd,&frecv,sizeof(Frame),0,(struct sockaddr*)&server,&len);
if(frecv.packet.data[0]=='e'){
```

```
printf("[-] Ack not received\n");
ack_rcv = 0;
}else{
if(frecv.sq == 0 && frecv.ack == frame_id+1){
printf("[+] Ack received\n");
ack_rcv = 1;
}
else{
printf("[-] Ack not received\n");
ack_rcv = 0;
}
}
frame_id++;

}
close(sockfd);
}
```

EXPECTED OUTPUT

Server

Server is running on port 8888...

Received Frame: seq=0, kind=1, expected=0

[+] Frame received: Hello

[+] ACK sent for seq 0

Received Frame: seq=1, kind=1, expected=1

[+] Frame received: World

[-] Simulating ACK loss

Received Frame: seq=1, kind=1, expected=1

[+] Frame received: World

[+] ACK sent for seq 1

Client

Enter data for frame 0: Hello

[+] Frame 0 sent

[+] ACK received for frame 0

Enter data for frame 1: World

[+] Frame 1 sent

[-] ACK not received, resending frame 1

[+] Frame 1 sent

[+] ACK received for frame 1

MODEL VIVA QUESTIONS

1. What is Stop-and-Wait ARQ?
2. What does ARQ stand for?
3. How many frames can be in transit at a time?
4. What is the difference between Stop-and-Wait and Sliding Window?

GO BACK N

ALGORITHM

Server

1. Start the program.
2. Create a TCP socket.
3. Set server IP (INADDR_ANY) and port (8084).
4. Bind the socket to the server address.
5. Listen for incoming client connections.
6. Accept a client connection.
7. Repeat indefinitely:
 8. Read 1 byte for frame type (ch) and 1 byte for sequence number (g).
 9. If ch == 'd' (data frame):
 1. Generate a random number to simulate corruption.
 2. If frame is corrupted and g == f+1:
 1. Print "Data <g> Corrupted".
 2. Send NACK (c) and sequence number to client.
 3. Else if frame is correct and g == f+1:
 1. Print "Data <g> received".
 2. Send ACK (a) and sequence number to client.
 3. Update last acknowledged frame f = g.
 4. Else:
 1. Print "Discarded data <g>".
 10. Go back to step 7.
 11. End program.

Client

1. Start the program.
2. Create a TCP socket.
3. Set server IP and port (8084).
4. Connect to the server.
5. Create two threads:
6. **senddata():**
 1. Increment sequence number s.
 2. Send frame type 'd' and sequence number s to server.
 3. Print "Send data <s>".
7. **receivedata():**
 1. Receive ACK/NACK (ca) and sequence number g.
 2. If ACK ('a'), print "Ack <g> Received".
 3. If NACK ('c'), print "Sending again" and set s = g-1 to retransmit.
8. Keep threads running indefinitely.
9. End program.

PROGRAM**Server**

```
#include<stdio.h>

#include<netdb.h>

#include<pthread.h>

main(){

    int sockfd, newsockfd, clilen, g = 0, n, m, p = 0, f = -1,k;

    char ca = 'a', can = 'c', ch;

    struct sockaddr_in serv_addr, cli_addr;
```

```
serv_addr.sin_family = AF_INET;

serv_addr.sin_addr.s_addr = INADDR_ANY;

serv_addr.sin_port = 8084;

sockfd = socket(PF_INET,SOCK_STREAM,0);

bind(sockfd,(struct sockaddr*)&serv_addr,sizeof(serv_addr));

listen(sockfd,5);

printf("Server is waiting.....\n");

clilen = sizeof(cli_addr);

newsockfd = accept(sockfd,(struct sockaddr *)&cli_addr,&clilen);

while(1){

    n = read(newsockfd,&ch,1);

    m = read(newsockfd,&g,1);

    if(ch == 'd'){

        k = rand();

        if (k%3 == 0){

            if(g == f+1){

                printf("Data %d Corrupted\n",g);

                write(newsockfd,&can,1);

                write(newsockfd,&g,1);

                sleep(2);

            }

            else{

                printf("Discarded data %d\n",g);

            }

        }

    }

}
```

```
        else{

                p = f + 1;

                if(g == p){

                        printf("Data %d received\n",g);

                        write(newsockfd,&ca,1);

                        write(newsockfd,&g,1);

                        printf("Data %d acknowledged\n",g);

                        f = g;

                }

                else

                        printf("Discarded data %d\n",g);

                sleep(2);

        }

    }

}
```

Client

```
#include<stdio.h>

#include<netdb.h>

#include<unistd.h>

#include<pthread.h>

int sockfd,m,n,s=-1,g=0;

void* senddata(){

    char ch='d';

    while(1){
```

```
        s++;

        write(sockfd,&ch,1);

        write(sockfd,&s,1);
        printf("Send data %d\n",s);
        sleep(1);
    }
}

void* receivedata(){
    char ca;
    while(1){
        n=read(sockfd,&ca,1);
        m=read(sockfd,&g,1);
        if(ca=='a'){
            printf("Ack %d Received\n",g);
            sleep(1);
        }
        else if(ca=='c'){
            printf("Sending again\n");
            s=g-1;
        }
    }
}

main(){
    pthread_t s,r;
```

```
struct sockaddr_in cliaddr;  
  
cliaddr.sin_family=AF_INET;  
  
cliaddr.sin_addr.s_addr= INADDR_ANY;  
  
cliaddr.sin_port= 8084;  
  
sockfd=socket(PF_INET,SOCK_STREAM,0);  
  
connect(sockfd,(struct sockaddr*)&cliaddr,sizeof(cliaddr));  
  
printf("Connecting to server.....\n");  
  
pthread_create(&s,0,senddata,0);  
  
sleep(1);  
  
pthread_create(&r,0,receivedata,0);  
  
sleep(1);  
  
while(1);}
```

EXPECTED OUTPUT

Server

Server is waiting.....

Data 0 received

Data 0 acknowledged

Data 1 received

Data 1 acknowledged

Data 2 Corrupted

Data 3 received

Data 3 acknowledged

Discarded data 2

Data 4 received

Data 4 acknowledged

Client

Connecting to server.....

Send data 0

Ack 0 Received

Send data 1

Ack 1 Received

Send data 2

Sending again

Send data 2

Ack 2 Received

Send data 3

Ack 3 Received

Send data 4

MODEL VIVA QUESTIONS

1. What is Go-Back-N ARQ?
2. What happens when a frame is lost or damaged?
3. How does Go-Back-N improve efficiency over Stop-and-Wait?
4. Which layer uses Go-Back-N?
5. Compare Go-Back-N and Stop-and-Wait.

SELECTIVE REPEAT ARQ

ALGORITHM

Server

1. Start the program.
2. Create a TCP socket.
3. Set server IP (INADDR_ANY) and port (e.g., 8000).
4. Bind the socket to the server address.
5. Listen for incoming client connections.
6. Accept a client connection.
7. Repeat indefinitely:
 8. Read 1 byte for frame type (ch) and 1 byte for sequence number (g).
 9. If ch == 'd' (data frame):
 1. Generate a random number to simulate corruption.
 2. If the frame is corrupted (rand()%3 == 0):
 1. Print "Data <g> corrupted".
 2. Send NACK (c) with sequence number g to client.
 3. Else (frame received correctly):
 1. Print "Data received: <g>".
 2. Send ACK (a) with sequence number g to client.
 3. Update last correctly received frame f = g.
 10. Frames are **individually acknowledged**, so out-of-order frames are not discarded (Selective Repeat).
 11. Go back to step 7.
 12. End program.

Client

1. Start Program
 2. Initialize Variables
 1. WSIZE = 2 (window size)
 2. up = -1 (last sent frame number)
 3. low = 0 (last acknowledged frame)
 4. Create a TCP socket (sockfd) and connect to the server.
 3. Create Two Threads
 1. Sender Thread
 2. Receiver Thread
- Sender Thread (senddata)
4. Loop forever:
 1. Increment up (next frame number to send).
 2. If number of unacknowledged frames exceeds window size ($up - low > WSIZE$):
 1. Print "congestion...".
 2. Wait for 1 second.
 3. Print "Data sent : up".
 4. Send a data character 'd' and frame number up to the receiver.
 5. Sleep for 1 second.

Receiver Thread (recvdata)

5. Loop forever:
 1. Read a character c and an integer g from the socket.
 2. If c == 'a' (acknowledgment received):
 1. Print "Data acknowledged : g".
 2. Increment low to slide the window.

3. Else if c == 'c' (data corrupted):
 1. Print "Data g corrupted, sending again".
 2. Resend 'd' and packet number g.
4. Sleep for 1 second.

Main Program

6. Start both threads.
7. Keep the program running indefinitely.

PROGRAM

Server

```
#include<stdio.h>

#include<sys/types.h>

#include<arpa/inet.h>

#include<pthread.h>

int main()

{

    int g=0,p,f=-1;

    char ch,ca = 'a', can = 'c';

    struct sockaddr_in server,client;

    int len = sizeof(client);

    server.sin_family = AF_INET;

    server.sin_port = 8000;

    server.sin_addr.s_addr = INADDR_ANY;

    int sockfd = socket(AF_INET,SOCK_STREAM,0);

    bind(sockfd,(struct sockaddr*)&server,sizeof(server));

    listen(sockfd,5);

    printf("Server is waiting.....\n");
```

```
int newfd = accept(sockfd,(struct sockaddr*)&client,&len);

while(1){

    read(newfd,&ch,1);

    read(newfd,&g,1);

    if(ch == 'd'){

        int k = rand();

        if(k%3==0){

            printf("Data %d corrupted\n",g);

            write(newfd,&can,1);

            write(newfd,&g,1);

            sleep(2);

        }else{

            printf("Data received : %d\n",g);

            write(newfd,&ca,1);

            write(newfd,&g,1);

            printf("Data acknowledged : %d\n",g);

            f = g;

            sleep(2);

        }

    }

}
```

Client

```
#include<stdio.h>

#include<pthread.h>

#include<netdb.h>

#define WSIZE 2

int sockfd,up=-1,low=0,g;

void* senddata(){

    char ch = 'd';

    while(1){

        up++;

        while((up-low)>WSIZE){

            printf("congestion....\n");

            sleep(1);

        }

        printf("Data sent : %d\n",up);

        write(sockfd,&ch,1);

        write(sockfd,&up,1);

        sleep(1);

    }

}

void* recvdata(){

    char c;

    char ch = 'd';

    while(1){

        read(sockfd,&c,1);
```

```
        read(sockfd,&g,1);

        if(c=='a'){

            printf("Data acknowledged : %d\n",g);

            low++;

        }

        else if(c=='c'){

            printf("Data %d corrupted, sending again\n",g);

            write(sockfd,&ch,1);

            write(sockfd,&g,1);

        }

        sleep(1);

    }

}

int main()

{

    pthread_t s,r;

    sockfd = socket(AF_INET,SOCK_STREAM,0);

    struct sockaddr_in server;

    server.sin_family = AF_INET;

    server.sin_port = 8000;

    server.sin_addr.s_addr = INADDR_ANY;

    connect(sockfd,(struct sockaddr*)&server,sizeof(server));

    pthread_create(&s,0,senddata,0);

    sleep(1);

    pthread_create(&r,0,recvdata,0);

    sleep(1);
```

```
        while(1);  
    }
```

EXPECTED OUTPUT

Server

Server is waiting.....

Data received : 0

Data acknowledged : 0

Data received : 1

Data acknowledged : 1

Data 2 corrupted

Data received : 2

Data acknowledged : 2

Data received : 3

Data acknowledged : 3

Client

Data sent : 0

Data acknowledged : 0

Data sent : 1

Data acknowledged : 1

Data sent : 2

Data 2 corrupted, sending again

Data sent : 3 ← May appear before ack due to window

Data acknowledged : 2

Data acknowledged : 3

Data sent : 4

Data acknowledged : 4

MODEL VIVA QUESTIONS

1. What is Selective Repeat ARQ?
2. What happens when a frame is lost?
3. Compare Selective Repeat and Go-Back-N.

RESULT

The Stop-and-Wait, Go-Back-N, and Selective Repeat ARQ protocols were successfully implemented and their reliability and efficiency characteristics were observed.

EXP NO: 6**DISTANCE VECTOR****AIM**

To study and implement the Distance Vector Routing algorithm

LEARNING OBJECTIVE

- To understand the concept of dynamic routing in computer networks.
- To study the working principle of the Distance Vector Routing algorithm.
- To learn how routers exchange routing information with neighboring nodes.
- To determine the shortest path between network nodes using distance vectors.

DESCRIPTION

Distance Vector Routing is a dynamic routing technique used in computer networks to determine the shortest path between nodes. In this approach, each router maintains a routing table containing the distance (cost) to reach every other node in the network along with the next hop. Routers periodically exchange their routing tables with neighboring routers and update their own tables based on the received information using the Bellman–Ford algorithm.

The routing table is updated by comparing the current distance with the newly calculated distance via neighboring nodes and selecting the minimum cost path. This process continues until all routers reach a stable state, known as convergence. Distance Vector Routing is simple to implement but may suffer from problems such as routing loops and slow convergence, which are addressed using techniques like split horizon and poisoned reverse.

ALGORITHM

1. Declare a structure node containing two arrays:
2. dist[] to store the distance to all other nodes
3. from[] to store the next hop router
4. Read the number of nodes in the network.
5. Read the cost matrix representing the cost between each pair of nodes.
6. Initialize:

7. Set the distance of each node to itself as zero.
8. Copy the cost matrix values into the distance table (rt[i].dist[j]).
9. Initialize the next hop (rt[i].from[j]) as the destination node itself.
10. Repeat the following steps until no further updates occur:
11. Set a counter to zero.
12. For each router i, destination j, and intermediate router k:
 1. Check if the current distance from i to j is greater than the distance from i to k plus the distance from k to j.
 2. If true:
 1. Update the distance table with the shorter distance.
 2. Update the next hop router.
 3. Increment the counter.
13. Stop the iteration when no distance updates occur (counter equals zero).
14. Display the routing table for each router showing:
15. Destination node
16. Next hop router
17. Minimum distance
18. End the program.

PROGRAM

```
#include<stdio.h>

struct node
{
    unsigned dist[20];
    unsigned from[20];
}rt[10];

int main(){
```

```
int costmat[20][20];

int nodes, i, j, k, count = 0;

printf("\nEnter the number of nodes : ");

scanf("%d",&nodes);

printf("\nEnter the cost matrix : \n");

for(i = 0; i<nodes; i++){

    for(j = 0; j<nodes; j++){

        scanf("%d",&costmat[i][j]);

        costmat[i][i] = 0;

        rt[i].dist[j] = costmat[i][j]; //initiate the distance equal to cost matrix

        rt[i].from[j] = j; //initiate the source node

    }

}

do{

    count = 0;

    //we choose an arbitrary vertex k and we calculate

    //the direct distance from node i to k using cost matrix

    //and add the distance from node k to node j

    for(i = 0; i<nodes; i++){

        for(j = 0; j<nodes; j++){

            for(k = 0; k<nodes; k++){

                if(rt[i].dist[j] > costmat[i][k] + rt[k].dist[j]){

                    rt[i].dist[j] = rt[i].dist[k] + rt[k].dist[j];

                    rt[i].from[j] = k;

                    count++;

                }

            }

        }

    }

}
```

```

        }

    }

}

}while(count!=0);

for(i = 0; i<nodes; i++){

    printf("\n\n For router %d\n",i+1);

    for(j = 0; j<nodes ;j++)

        printf("\nnode %d via %d distance %d",j+1 ,rt[i].from[j]+1, rt[i].dist[j]);

    }

}

```

EXPECTED OUTPUT

Enter the cost matrix :

```

0
2
999
1
2
0
3
999
999
3
0
2
1
999

```

2

0

For router 1

node 1 via 1 distance 0

node 2 via 2 distance 2

node 3 via 4 distance 3

node 4 via 4 distance 1

For router 2

node 1 via 1 distance 2

node 2 via 2 distance 0

node 3 via 3 distance 3

node 4 via 1 distance 3

For router 3

node 1 via 4 distance 3

node 2 via 2 distance 3

node 3 via 3 distance 0

node 4 via 4 distance 2

For router 4

node 1 via 1 distance 1

node 2 via 1 distance 3

node 3 via 3 distance 2

node 4 via 4 distance 0

MODEL VIVA QUESTIONS

What is Distance Vector Routing?

Which algorithm is used in Distance Vector Routing?

What is a routing table?

Which routing protocols use Distance Vector Routing?

What is the initial state of a routing table?

RESULT

The Distance Vector Routing algorithm was successfully implemented and verified.

EXP NO: 7

FILE TRANSFER PROTOCOL

AIM

To implement the working of File Transfer Protocol (FTP) for transferring files between a client and a server.

LEARNING OBJECTIVE

- To understand the purpose and working of File Transfer Protocol (FTP).
- To study the client–server architecture used in FTP.
- To analyze the role of control and data connections in FTP.
- To observe reliable file transfer using TCP.

DESCRIPTION

File Transfer Protocol (FTP) is a standard application layer protocol used to transfer files between a client and a server over a TCP/IP network. It follows a client–server architecture and provides reliable communication for uploading, downloading, and managing files on a remote system.

Characteristics of FTP

- FTP uses TCP at the transport layer.
- FTP uses port number 21 for control connection.
- FTP uses port number 20 for data connection.
- FTP uses persistent TCP connections for control connection.
- FTP uses non-persistent connections for data connection.
- FTP is a connection oriented protocol.
- FTP is an out-of-band protocol as data and control information flow over different connections.

ALGORITHM**server**

1. Start the program.
2. Create a TCP socket.
3. Bind the socket to port 8080 and any IP address.
4. Listen for incoming client connections.
5. Accept a client connection.
6. Receive the filename from the client.
7. Open the requested file.
8. If the file exists, send its contents to the client; otherwise, send an error message.
9. Close the client connection.
10. Repeat from step 5 for the next client.

Client

1. Start the program.
2. Create a TCP socket.
3. Specify the server IP address and port number (8080).
4. Connect the client socket to the server.
5. Read the filename from the user.
6. Send the filename to the server.
7. Receive data from the server and display it.
8. Close the client socket.
9. Stop the program.

PROGRAM**Server**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
```

```
#define BUFFER_SIZE 1024
void handle_client(int client_socket)
{
    char buffer[BUFFER_SIZE] = {0};
    char filename[BUFFER_SIZE] = {0};
    int bytes_received = recv(client_socket, filename, BUFFER_SIZE, 0);
    if (bytes_received < 0) {
        perror("Error receiving filename");
        close(client_socket);
        return;
    }

    FILE *file = fopen(filename, "r");
    if (file == NULL)
    {
        char *error_message = "File not found";
        send(client_socket, error_message, strlen(error_message), 0);
        perror("File not found");
    }
    else
    {
        while (fgets(buffer, BUFFER_SIZE, file) != NULL)
        {
            send(client_socket, buffer, strlen(buffer), 0);
            memset(buffer, 0, BUFFER_SIZE);
        }
        printf("File Send to client\n");
        fclose(file);
    }
    close(client_socket);
}

int main()
{
    int server_socket, client_socket;
```

```
struct sockaddr_in server_addr, client_addr;

socklen_t addr_len = sizeof(client_addr);
server_socket = socket(AF_INET, SOCK_STREAM, 0);
if (server_socket == 0)
{
    perror("Socket failed");
    exit(EXIT_FAILURE);
}
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(PORT);
if (bind(server_socket, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
    perror("Bind failed");
    exit(EXIT_FAILURE);
}
if (listen(server_socket, 3) < 0) {
    perror("Listen failed");
    exit(EXIT_FAILURE);
}

printf("Server is listening on port %d\n", PORT);
while (1) {
    client_socket = accept(server_socket, (struct sockaddr *)&client_addr, &addr_len);
    if (client_socket < 0) {
        perror("Accept failed");
        exit(EXIT_FAILURE);
    }
    printf("Client connected\n");
    handle_client(client_socket);
}
close(server_socket);
return 0;
}
```

Client

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define BUFFER_SIZE 1024
int main()
{
    int client_socket;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE] = {0};
    char filename[BUFFER_SIZE] = {0};
    client_socket = socket(AF_INET, SOCK_STREAM, 0);
    if (client_socket < 0)
    {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    if (connect(client_socket, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
        perror("Connection failed");
        exit(EXIT_FAILURE);
    }
    printf("Enter the filename to request: ");
    scanf("%s", filename);
    send(client_socket, filename, strlen(filename), 0);
    printf("File content received:\n");
    while (recv(client_socket, buffer, BUFFER_SIZE, 0) > 0) {
        printf("%s", buffer);
        memset(buffer, 0, BUFFER_SIZE);
    }
}
```

```
printf("\n");  
close(client_socket);return 0;  
}
```

EXPECTED OUTPUT

Server

Server is listening on port 8080

Client connected

File Send to client

Client

Enter the filename to request: test

File content received:

hello all

“This is a implementation of File transfer protocol”

MODEL VIVA QUESTIONS

1. What is FTP?
2. What is the difference between **FTP** and **SFTP**?
3. What is the default port number used by FTP?
4. What is the role of the FTP server?
5. Mention any two applications of FTP.

RESULT

The File Transfer Protocol (FTP) was successfully implemented and tested.

EXP NO: 8**SIMPLE MAIL TRANSFER PROTOCOL****AIM**

To study and implement the Simple Mail Transfer Protocol (SMTP) and understand the process of email transmission between mail clients and mail servers.

LEARNING OBJECTIVE

- To understand the purpose and role of Simple Mail Transfer Protocol in email communication.
- To study the client–server architecture used in SMTP.
- To learn the basic SMTP commands used for sending emails.
- To analyze the process of email transmission between mail clients and mail servers.
- To understand the use of TCP for reliable mail delivery.
- To gain practical experience in sending emails using SMTP.

DESCRIPTION

Simple Mail Transfer Protocol (SMTP) is an application layer protocol used for sending electronic mail over the Internet or a TCP/IP network. It is responsible for transferring email messages from a sender's mail client to the sender's mail server and between mail servers until the message reaches the recipient.

SMTP follows a client–server model, where the sending mail server or client acts as the SMTP client and the receiving mail server acts as the SMTP server. Communication takes place over a reliable TCP connection, commonly using port 25, 587, or 465. SMTP uses a set of text-based commands and responses to control the email transmission process.

SMTP Control Keywords / Commands

- HELO – Identifies the sending client to the mail server
- EHLO – Extended version of HELO (used with ESMTP)
- MAIL FROM: – Specifies the sender's email address
- RCPT TO: – Specifies the recipient's email address
- DATA – Indicates the start of the email message content

- RSET – Resets the current mail transaction
- NOOP – No operation; keeps the connection alive
- VRFY – Verifies whether a mailbox exists on the server
- QUIT – Terminates the SMTP session

SMTP Response Codes

- 220 – Server ready
- 250 – OK
- 354 – Start message input
- 221 – Closing connection
- 550 – Mailbox unavailable

ALGORITHM

Server

1. Start the program.
2. Read the port number from the command-line argument.
3. Create a UDP socket using socket().
4. Initialize the server address structure with IP address and port number.
5. Bind the socket to the server address using bind().
6. Wait for a message from the client using recvfrom().
7. Receive the client's request and send a greeting message (220).
8. Receive the HELO command from the client and send response 250 OK.
9. Receive the MAIL FROM command and send response 250 OK.
10. Receive the RCPT TO command and send response 250 OK.
11. Receive the DATA command and send response 354 Go ahead.
12. Receive the mail body from the client.
13. Receive the QUIT command from the client.
14. Send the termination response 221 OK.

15. Close the socket.

16. Stop the program.

Client

1. Start the program.

2. Read the server IP address and port number from command-line arguments.

3. Create a UDP socket using `socket()`.

4. Initialize the server address structure.

5. Send an initial greeting message to the server.

6. Receive the server greeting code (220).

7. Send the HELO command to the server.

8. Receive the server response (250 OK).

9. Read the sender's email address and send MAIL FROM command.

10. Receive the server response (250 OK).

11. Read the recipient's email address and send RCPT TO command.

12. Receive the server response (250 OK).

13. Send the DATA command to the server.

14. Receive the server response (354 Go ahead).

15. Read the mail body from the user and send it to the server.

16. Send the QUIT command to the server.

17. Receive the termination response (221 OK).

18. Close the socket.

19. Stop the program.

PROGRAM**Server**

```
#include<stdio.h>

#include<string.h>

#include<sys/socket.h>

#include<arpa/inet.h>

#include<stdlib.h>

#include<sys/types.h>

#include<netinet/in.h>

#include<unistd.h>

#define BUF_SIZE 256

int main(int argc,char* argv[])

{

    struct sockaddr_in server,client;

    char str[50],msg[20];

    if(argc!=2)

        printf("Input format not correct");

    int sockfd=socket(AF_INET,SOCK_DGRAM,0);

    if(sockfd==-1)

        printf("Error in socket()");

    server.sin_family=AF_INET;

    server.sin_addr.s_addr=INADDR_ANY;

    server.sin_port=htons(atoi(argv[1]));

    client.sin_family=AF_INET;

    client.sin_addr.s_addr=INADDR_ANY;
```

```
client.sin_port=htons(atoi(argv[1]));  
if(bind(sockfd,(struct sockaddr *)&server,sizeof(server))<0)  
printf("Error in bind()! \n");  
socklen_t client_len=sizeof(client);  
printf("server waiting.....");  
sleep(3);  
if(recvfrom(sockfd,str,100,0,(struct sockaddr *)&client,&client_len)<0)  
printf("Error in recvfrom()!");  
printf("\nGot message from client : %s",str);  
printf("\nSending greeting message to client : ");  
strcpy(str,"220 127.0.0.1");  
sleep(10);  
if(sendto(sockfd,str,sizeof(str),0,(struct sockaddr *)&client,sizeof(client))<0)  
printf("Error in send");  
sleep(3);  
if((recvfrom(sockfd,str,sizeof(str),0,(struct sockaddr *)&client,&client_len)),0)  
printf("Error in recv");  
if(strncmp(str,"HELO",4))  
printf("\n'HELO' expected from client....");  
printf("\n%s",str);  
printf("\nSending response...");  
strcpy(str,"250 ok");  
if(sendto(sockfd,str,sizeof(str),0,(struct sockaddr *)&client,sizeof(client))<0)  
printf("Error in send");  
sleep(3);
```

```
if((recvfrom(sockfd,str,sizeof(str),0,(struct sockaddr *)&client,&client_len))<0)

printf("Error in recv");

if(strncmp(str,"MAIL FROM : ",9))

printf("MAIL FROM expected from client...");

printf("\n%s",str);

printf("\nSending response....");

strcpy(str,"250 ok");

if(sendto(sockfd,str,sizeof(str),0,(struct sockaddr *)&client,sizeof(client))<0)

printf("Error in send");

sleep(3);

if((recvfrom(sockfd,str,sizeof(str),0,(struct sockaddr *)&client,&client_len))<0)

printf("Error in recv");

if(strncmp(str,"RCPT TO",7))

printf("\nRCPT TO expected from client....");

printf("\n%s",str);

printf("\nSending response....");

strcpy(str,"250 ok");

if(sendto(sockfd,str,sizeof(str),0,(struct sockaddr *)&client,sizeof(client))<0)

printf("Error in send");

sleep(3);

if((recvfrom(sockfd,str,sizeof(str),0,(struct sockaddr *)&client,&client_len))<0)

printf("Error in recv");

if(strncmp(str,"DATA",4))

printf("\nDATA expected from client....");

printf("\n %s",str);
```

```
printf("\nSending response....");
strcpy(str," 354 Go ahead");
if(sendto(sockfd,str,sizeof(str),0,(struct sockaddr *)&client,sizeof(client))<0)
printf("Error in send");
if((recvfrom(sockfd,msg,sizeof(str),0,(struct sockaddr *)&client,&client_len))<0)
printf("Error in recv");
printf(" mail body received");
printf("\n%s",msg);
if((recvfrom(sockfd,str,sizeof(str),0,(struct sockaddr *)&client,&client_len))<0)
printf("Error in recv");
if(strncmp(str,"QUIT",4))
printf("quit expected from client....");
printf("\nSending quit...");
strcpy(str," 221 OK");
if(sendto(sockfd,str,sizeof(str),0,(struct sockaddr *)&client,sizeof(client))<0)
printf("Error in send");
close(sockfd);
return 0;
}
```

Client

```
#include<string.h>
#include<sys/socket.h>
#include<netdb.h>
#include<stdlib.h>
#include<stdio.h>
```

```
#include<netinet/in.h>

#include<sys/types.h>

#include<netinet/in.h>

#include<unistd.h>

#define BUF_SIZE 256

int main(int argc,char* argv[])
{
    struct sockaddr_in server,client;

    char str[50]="hi";

    char mail_f[50],mail_to[50],msg[20],c;

    int t=0;

    socklen_t l=sizeof(server);

    if(argc!=3)

        printf("Input format not correct");

    int sockfd=socket(AF_INET,SOCK_DGRAM,0);

    if(sockfd===-1)

        printf("Error in socket()");

    server.sin_family=AF_INET;

    server.sin_addr.s_addr=INADDR_ANY;

    client.sin_port=htons(atoi(argv[2]));

    printf("Sending hi to server");

    sleep(10);

    if(sendto(sockfd,str,sizeof(str),0,(struct sockaddr*)&server,sizeof(server))<0)

        printf("Error in sento");

    if(recvfrom(sockfd,str,sizeof(str),0,(struct sockaddr *)&server,&l)<0)
```

```
printf("Error in recv");
printf("\ngreeting msg is %s",str);
if(strncmp(str,"220",3))
printf("\nConn not established \n code 220 expected");
printf("\nSending HELO");
strcpy(str,"HELO 127.0.0.1");
if(sendto(sockfd,str,sizeof(str),0,(struct sockaddr *)&server,sizeof(server))<0)
printf("Error in sendto");
sleep(3);
printf("\nReceiving from server");
if((recvfrom(sockfd,str,sizeof(str),0,(struct sockaddr *)&server,&l))<0)
printf("Error in recv");
if(strncmp(str,"250",3))
printf("\nOk not received from server");
printf("\nServer has send %s",str);
printf("\nEnter FROM address : \t");
scanf("%s",mail_f);
strcpy(str,"MAIL FROM");
strcat(str,mail_f);
if(sendto(sockfd,str,sizeof(str),0,(struct sockaddr *)&server,sizeof(server))<0)
printf("Error in sendto");
sleep(3);
if((recvfrom(sockfd,str,sizeof(str),0,(struct sockaddr *)&server,&l))<0)
printf("Error in recv");
if(strncmp(str,"250",3))
```

```
printf("\nOk not received from server");

printf("%s",str);

printf("\nEnter TO address : \t");

scanf("%s",mail_to);

strcpy(str,"RCPT TO");

strcat(str,mail_to);

if(sendto(sockfd,str,sizeof(str),0,(struct sockaddr *)&server,sizeof(server))<0)

printf("Error in sendto");

sleep(3);

if((recvfrom(sockfd,str,sizeof(str),0,(struct sockaddr *)&server,&l))<0)

printf("Error in recv");

if(strncmp(str,"250",3))

printf("\nOk not received from server");

printf("%s",str);

printf("\nSending DATA to server");

strcpy(str,"DATA");

if(sendto(sockfd,str,sizeof(str),0,(struct sockaddr *)&server,sizeof(server))<0)

printf("Error in sendto");

sleep(3);

if((recvfrom(sockfd,str,sizeof(str),0,(struct sockaddr *)&server,&l))<0)

printf("Error in recv");

if(strncmp(str,"354",3))

printf("\nOk not received from server");

printf("%s",str);

printf("\nEnter mail body: ");
```

```
while(1)
{
c=getchar();
if(c=='$')
{
msg[t]='\0';
break;
}
if(c=='\0')
continue;
msg[t++]=c;
}

if(sendto(sockfd,msg,sizeof(msg),0,(struct sockaddr *)&server,sizeof(server))<0)
printf("Error in sendto");

sleep(3);

printf("\nSending QUIT to server");

strcpy(str,"QUIT");

if(sendto(sockfd,str,sizeof(str),0,(struct sockaddr *)&server,sizeof(server))<0)
printf("Error in sendto");

if((recvfrom(sockfd,str,sizeof(str),0,(struct sockaddr *)&server,&l))<0)
printf("Error in recv");

if(strncmp(str,"221",3))

printf("\nOk not received from server");

printf("\nServer has send GOODBYE.....Closing conn\n");

printf("\n Bye");
```

```
close(sockfd);  
return 0;  
}
```

EXPECTED OUTPUT

```
student@administrator:~$ ./serv 5600  
server waiting.....  
Got message from client : hi  
Sending greeting message to client :  
HELO 127.0.0.1  
Sending response...  
MAIL FROMabc@vidya  
Sending response....  
RCPT TOxyz@vidya  
Sending response....  
DATA  
Sending response.... mail body received  
hi  
how r u?  
thank y  
Sending quit...  
./cli 127.0.0.1 5600  
Sending hi to server  
greeting msg is 220 127.0.0.1  
Sending HELO  
Receiving from server
```

Server has send 250 ok

Enter FROM address : abc@vidya

250 ok

Enter TO address : xyz@vidya

250 ok

Sending DATA to server

Ok not received from server 354 Go ahead

Enter mail body: hi

how r u?

thank you\$

Sending QUIT to server

Ok not received from server

Server has send GOODBYE.....Closing conn

MODEL VIVA QUESTIONS

1. What is SMTP?
2. What are SMTP commands?
3. Explain the use of the HELO command.
4. What does SMTP reply code 250 indicate?
5. What does reply code 354 mean?

RESULT

The Simple Mail Transfer Protocol (SMTP) was successfully implemented and verified.

EXP NO: 9**LEACKY BUCKET****AIM**

To simulate the Leaky Bucket algorithm for congestion control in computer networks.

LEARNING OBJECTIVE

- To understand the concept of congestion control in computer networks.
- To study the working principle of the Leaky Bucket algorithm.
- To analyze how traffic is regulated using a fixed output rate.
- To observe packet buffering and packet loss during congestion.
- To evaluate the effectiveness of the Leaky Bucket algorithm in smoothing bursty traffic.

DESCRIPTION

The **Leaky Bucket algorithm** is a traffic shaping and congestion control mechanism used in computer networks to regulate the flow of data packets. It works on the principle of a bucket with a fixed capacity that leaks data at a constant rate. Incoming packets are added to the bucket, and packets are transmitted at a steady output rate regardless of the bursty nature of the input traffic.

If the incoming packet rate exceeds the bucket capacity, excess packets are discarded, thereby preventing network congestion. This algorithm smoothens bursty traffic and ensures a uniform transmission rate, making it suitable for real-time and multimedia applications where consistent data flow is required.

ALGORITHM

1. Start the program.
2. Read the bucket size, output rate, and number of incoming packets.
3. Initialize the current bucket storage to zero.
4. Repeat until all packets are processed:
 - a. Read the size of the incoming packet.
 - b. If the incoming packet size is less than or equal to the available space in the bucket, add

it to the bucket.

c. Otherwise, discard the excess packets and fill the bucket to its maximum capacity.

d. Display the bucket status before outgoing transmission.

e. Transmit packets at the fixed output rate.

f. Update and display the bucket status after outgoing transmission.

5. Stop the program.

PROGRAM

```
#include<stdio.h>
```

```
int main(){
```

```
    int bsize, out, n, store = 0, in;
```

```
    printf("Enter bucket size, output rate, no of inputs: ");
```

```
    scanf("%d%d%d", &bsize, &out, &n);
```

```
    while(n!=0){
```

```
        printf("\nEnter incoming packet size: ");
```

```
        scanf("%d", &in);
```

```
        if(in<=(bsize-store)){
```

```
            store += in;
```

```
            printf("Before outgoing, bucket size %d out of %d", store , bsize);
```

```
        }else{
```

```
            printf("Incoming packets, %d discarded out of %d", in-(bsize-store), in);
```

```
            store = bsize;
```

```
            printf("Before outgoing, bucket size %d out of %d", store, bsize);
```

```
        }
```

```
        if(store>=out){
```

```
            store -= out;
```

```
        }else{
```

```
        store = 0;

    }

    printf("After outgoing, bucket size %d occupied of %d", store, bsize);

    n--;

}

}
```

EXPECTED OUTPUT

Enter bucket size, output rate, no of inputs: 10

4

5

Enter incoming packet size: 2

Before outgoing, bucket size 2 out of 10After outgoing, bucket size 0 occupied of 10

Enter incoming packet size: 5

Before outgoing, bucket size 5 out of 10After outgoing, bucket size 1 occupied of 10

Enter incoming packet size: 4

Before outgoing, bucket size 5 out of 10After outgoing, bucket size 1 occupied of 10

Enter incoming packet size: 3

Before outgoing, bucket size 4 out of 10After outgoing, bucket size 0 occupied of 10

Enter incoming packet size: 5

Before outgoing, bucket size 5 out of 10After outgoing, bucket size 1 occupied of 10

MODEL VIVA QUESTIONS

1. Explain the working principle of the Leaky Bucket algorithm.
2. What happens when the bucket overflows?
3. What is meant by bucket size in the Leaky Bucket algorithm?
4. What is output rate in the Leaky Bucket algorithm?

5. How does the Leaky Bucket algorithm control congestion?

RESULT

Leaky Bucket algorithm was successfully implemented and verified.

EXP NO: 10**UNDESTANDING THE WIRESHARK TOOL****AIM**

To study and analyze network packets using the Wireshark tool in order to understand network communication and protocol behavior.

LEARNING OBJECTIVE

- To understand the purpose and features of the Wireshark tool.
- To learn how to capture live network traffic.
- To analyze packets at different protocol layers.
- To apply filters to isolate specific network traffic.
- To observe the working of common network protocols.
- To gain hands-on experience in network packet analysis.

DESCRIPTION**Steps to Use Wireshark**

1. Install and launch the Wireshark application.
2. Select the appropriate **network interface** (Ethernet/Wi-Fi).
3. Click **Start Capture** to begin packet capturing.
4. Generate network traffic (open a website, ping a server, etc.).
5. Click **Stop Capture** after sufficient packets are captured.
6. Select any packet to view detailed information.
7. Expand protocol layers to analyze packet contents.
8. Apply **display filters** (e.g., tcp, udp, http) to narrow results.
9. Save the captured packets if required.

Advantages of Wireshark

- Free and open-source tool

- Supports a large number of protocols
- Provides detailed packet-level analysis
- Powerful filtering and search options
- Useful for learning, troubleshooting, and security analysis

EXPECTED OUTPUT

Example: TCP Packet

No.	Time	Source	Destination	Protocol	Length	Info
25	3.4521	192.168.1.5	142.250.195.78	TCP	74	443 → 51562 [SYN]

Packet Details:

- Source IP: 192.168.1.5
- Destination IP: 142.250.195.78
- Protocol: TCP
- Source Port: 51562
- Destination Port: 443
- Flags: SYN

MODEL VIVA QUESTIONS

1. What is Wireshark?
2. What type of tool is Wireshark?
3. Which OSI layers can be analyzed using Wireshark?
4. What is packet capturing?
5. Name the file format used by Wireshark to save captures.

RESULT

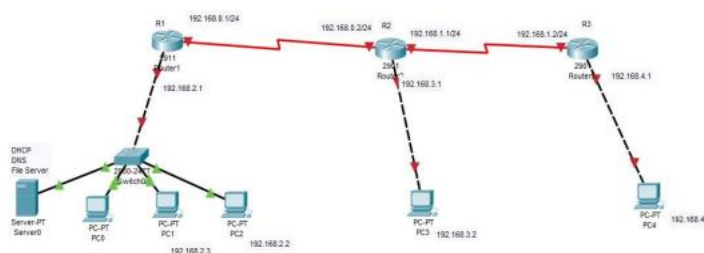
Wireshark tool was successfully used to capture and analyze live network traffic.

EXP NO: 11**DESIGN AND CONFIGURE NETWORK WITH MULTIPLE SUBNETS
USING REQUIRED NETWORK DEVICES**

To design and configure a network with multiple subnets with wired and wireless LANs using required network devices. Configure the following services in the network- TELNET, SSH, FTP server, Webserver, File server, DHCP server and DNS server.*

LEARNING OBJECTIVE

- Understand the basic principles of network design and configuration.
- Create and configure a **network topology** using appropriate networking tools.
- Configure **network services** such as TCP/UDP-based applications.
- Analyze **data flow, congestion, and performance** within the network.
- Observe and interpret **network behavior using simulation and visualization tools**.
- Gain hands-on experience in **verifying and validating network services**.

DESCRIPTION

This experiment focuses on the design and configuration of a computer network along with the implementation of essential network services. A network topology is created using a network simulation tool, where multiple nodes are interconnected using links with defined bandwidth, delay, and queue properties. Network services such as TCP and UDP-based applications are configured to operate over the designed network.

The above topology shows 3 subnets. The network is designed with server, hosts, switch and Routers.

Configuration

Subnet 1 contains 3 hosts. Hosts are configured with IP addresses 192.168.2.2, 192.168.2.3, 192.168.2.4 and the server with 192.168.2.100 . These hosts are connected to the fast Ethernet port of the router through the switch Server is configured with DHCP, DNS etc. Subnet 2 contains one host with IP address 192.168.3.2 Subnet 3 contains one host with IP address 192.168.4.2. Routers R1, R2, R3 are connected through serial cable (R1,R2,R3) We need to configure static routing in the routers to enable communication of hosts in different subnets

Router 1

Configure the router serial interface with IP address 192.168.0.1 255.255.255.0 Configure the fast Ethernet port with IP address 192.168.2.1 Configure the static route -ip route 192.168.1.0 255.255.255.0 s0/0 ip route 192.168.3.0 255.255.255.0 s0/0 ip route 192.168.4.0 255.255.255.0 s0/0

Router 2

Configure the router serial interface 0 with IP address 192.168.0.2 255.255.255.0 Configure the router serial interface 1 with IP address 192.168.1.1 255.255.255.0 Configure the fast Ethernet port with IP address 192.168.3.1 Configure the static route -ip route 192.168.2.0 255.255.255.0 s0/0 ip route 192.168.4.0 255.255.255.0 s0/1

Router 3

Configure the router serial interface with IP address 192.168.1.2 255.255.255.0 Configure the fast Ethernet port with IP address 192.168.4.1 Configure the static route -ip route 192.168.0.0 255.255.255.0 s0/0 ip route 192.168.1.0 255.255.255.0 s0/0 ip route 192.168.2.0 255.255.255.0 s0/0

EXPECTED OUTPUT

Files and services can be accessed across subnets.

MODEL VIVA QUESTIONS

1. What is meant by network design?
2. What are the basic components required to design a network?
3. What is a network topology?
4. Name different types of network topologies.
5. Which topology is most commonly used in LANs? Why?

RESULT

The design and configuration of the network was carried out successfully.

EXP NO: 12

STUDY OF NS2 SIMULATOR

AIM

To simulate a network topology using NS2 and analyze the behavior of **TCP (FTP)** and **UDP**

LEARNING OBJECTIVE

- To understand the working of the NS2 simulator.
- To create and simulate network topologies using TCL scripts.
- To analyze TCP and UDP traffic behavior.
- To study congestion and queue behavior in a network.
- To visualize packet flow using NAM animation.

SOFTWARE REQUIREMENTS

- Operating System: Linux / Ubuntu
- Simulator: NS2
- Tools: NAM (Network Animator),

DESCRIPTION

NS2 (Network Simulator–2) is an open-source, discrete-event network simulator widely used for academic and research purposes.

It supports simulation of:

- TCP, UDP
- FTP, CBR traffic
- Routing and queue management
- Wired and wireless networks

NS2 uses **TCL scripting** to define network topology, traffic sources, and events.

The simulation produces:

- **Trace file (.tr)** for analysis

- **NAM file (.nam)** for animation

ALGORITHM

1. Initialize the NS2 simulator and open trace and NAM files.
2. Create network nodes and establish links between them.
3. Configure TCP (FTP) and UDP (CBR) traffic sources and sinks.
4. Set link properties, queue limits, and flow identifiers.
5. Schedule start and stop times for traffic generation.
6. Run the simulation and display animation using NAM.

PROGRAM

```
set ns [new Simulator]

#define color for data flows

$ns color 1 Blue

$ns color 2 Red

#open tracefiles

set tracefile1 [open out.tr w]

set winfile [open winfile w]

$ns trace-all $tracefile1

#open nam file

set namfile [open out.nam w]

$ns namtrace-all $namfile

#define the finish procedure

proc finish {} {

global ns tracefile1 namfile

$ns flush-trace

close $tracefile1

close $namfile
```

```
exec nam out.nam &

exit 0

}

#create six nodes

set n0 [$ns node]

set n1 [$ns node]

set n2 [$ns node]

set n3 [$ns node]

set n4 [$ns node]

set n5 [$ns node]

$n1 color Red

$n1 shape box

#create links between the nodes

$ns duplex-link $n0 $n2 2Mb 10ms DropTail

$ns duplex-link $n1 $n2 2Mb 10ms DropTail

$ns simplex-link $n2 $n3 0.3Mb 100ms DropTail

$ns simplex-link $n3 $n2 0.3Mb 100ms DropTail

set lan [$ns newLan "$n3 $n4 $n5" 0.5Mb 40ms LL Queue/DropTail MAC/Csma/Cd Channel]

#Give node position

$ns duplex-link-op $n0 $n2 orient right-down

$ns duplex-link-op $n1 $n2 orient right-up

$ns simplex-link-op $n2 $n3 orient right

$ns simplex-link-op $n3 $n2 orient left

#set queue size of link(n2-n3) to 20

$ns queue-limit $n2 $n3 20

#setup TCP connection
```

```
set tcp [new Agent/TCP/Newreno]

$ns attach-agent $n0 $tcp

set sink [new Agent/TCPSink/DelAck]

$ns attach-agent $n4 $sink

$ns connect $tcp $sink

$tcp set fid_ 1

$tcp set packet_size_ 552

#set ftp over tcp connection

set ftp [new Application/FTP]

$ftp attach-agent $tcp

#setup a UDP connection

set udp [new Agent/UDP]

$ns attach-agent $n1 $udp

set null [new Agent/Null]

$ns attach-agent $n5 $null

$ns connect $udp $null

$udp set fid_ 2

#setup a CBR over UDP connection

set cbr [new Application/Traffic/CBR]

$cbr attach-agent $udp

$cbr set type_ CBR

$cbr set packet_size_ 1000

$cbr set rate_ 0.01Mb

$cbr set random_ false

#scheduling the events

$ns at 0.1 "$cbr start"
```

```
$ns at 1.0 "$ftp start"

$ns at 124.0 "$ftp stop"

$ns at 125.5 "$cbr stop"

proc plotWindow {tcpSource file} {

global ns

set time 0.1

set now [$ns now]

set cwnd [$tcpSource set cwnd_]

puts $file "$now $cwnd"

$ns at [expr $now+$time] "plotWindow $tcpSource $file"

}

$ns at 0.1 "plotWindow $tcp $winfile"

$ns at 125.0 "finish"

$ns run
```

EXPECTED OUTPUT

After executing the TCL file, the following files are created:

- out.tr – Trace file recording all simulation events
- out.nam – NAM file for animation

MODEL VIVA QUESTIONS

1. What is NS2?
2. What is the use of TCL in NS2?
3. What is NAM?
4. What files are generated after running NS2?
5. What is the purpose of queue limit?

RESULT

The NS2 simulation was executed successfully.